DOCUMENTO TÉCNICO

SISTEMA-SAAS-DULCERIA

FASE 4: INFRAESTRUCTURA BASE COMPLETA + DESPLIEGUE LOCAL

Versión	4.1.0
Fecha	09 de Marzo de 2026
Autor	Bryan Stiven Bonilla Rosas
Estado	**OPERACIONAL**

Tecnología al servicio de las dulcerías colombianas

TABLA DE CONTENIDO

TABLA DE CONTENIDO 2
1. RESUMEN EJECUTIVO 4
1.1 ¿Qué hemos construido? 4
1.2 Estado Actual del Sistema 4
2. FASE ACTUAL DEL PROYECTO 5
2.1 Mapa de Fases del Proyecto 5
2.2 Hitos Alcanzados en Fase 4 5
3. ARQUITECTURA DEL SISTEMA 7
3.1 Diagrama Conceptual de Arquitectura 7
3.2 Redes Internas de Docker 7
3.3 Volúmenes Persistentes 8
4. COMPONENTES IMPLEMENTADOS 9
4.1 Backend – FastAPI (Python 3.12) 9
4.2 Frontend – React 18 + Vite 5.4 9
4.3 Base de Datos – PostgreSQL 15 + Redis 7 9
4.4 Monitoreo – Prometheus + Grafana 10
4.5 Proxy – Nginx 10
5. DETALLE DE LA CONSTRUCCIÓN 11
5.1 Estructura de Carpetas del Proyecto 11
5.2 Dockerización Detallada 12
5.3 Resolución de Problemas Documentados 12
6. CONFIGURACIÓN DE INFRAESTRUCTURA 14
6.1 Variables de Entorno (.env) 14
6.2 Configuración de Prometheus 14
7. COMANDOS DE OPERACIÓN 15
7.1 Script Principal sgdd.ps1 15
7.2 Comandos Manuales de Docker 15
7.3 Accesos Rápidos del Sistema 15
8. PRÓXIMOS PASOS Y HOJA DE RUTA 17
8.1 Fase 6 – Integración API-Frontend (Inmediato) 17
8.2 Fase 7 – Módulos de Negocio (Corto Plazo) 17
8.3 Fase 8 – Testing y QA (Mediano Plazo) 17
8.4 Fase 9 – Producción y Escalado (Largo Plazo) 17

9. ANEXOS TÉCNICOS.....19
Anexo A – Tabla Completa de Puertos del Sistema 19
Anexo B – Comandos de Diagnóstico 19
Anexo C – Guía de Solución de Problemas Comunes.....20
CONCLUSIÓN.....21

1. RESUMEN EJECUTIVO

1.1 ¿Qué hemos construido?

Hemos establecido la **infraestructura base completa** de un sistema SaaS multi-tenant para gestión de dulcerías. A continuación se detallan todos los componentes principales del sistema:

Componente	Tecnología	Descripción
Backend	FastAPI – Python 3.12	API REST con alta performance, documentación automática Swagger/OpenAPI
Frontend	React 18 + Vite 5.4	SPA moderna con hot reload para desarrollo ágil
Base de Datos	PostgreSQL 15	Base de datos relacional robusta con soporte multi-tenant
Caché	Redis 7	Caché en memoria y gestión de sesiones con persistencia
Monitoreo	Prometheus + Grafana	Recolección de métricas y dashboards visuales en tiempo real
Proxy	Nginx	Servidor web / proxy inverso con balanceo de carga
Contenerización	Docker + Docker Compose	7 servicios orquestados con redes y volúmenes aislados

1.2 Estado Actual del Sistema

Todos los servicios se encuentran en estado saludable y funcional en el entorno local de desarrollo:

Servicio	Estado	URL / Puerto
PostgreSQL	SALUDABLE	localhost:5432
Redis	SALUDABLE	localhost:6379
Backend API (FastAPI)	ACTIVO	http://localhost:8000
Frontend (React/Vite)	ACTIVO	http://localhost:5173
Grafana	ACTIVO	http://localhost:3000
Prometheus	ACTIVO	http://localhost:9090
Nginx (Proxy)	ACTIVO	http://localhost:80

2. FASE ACTUAL DEL PROYECTO

2.1 Mapa de Fases del Proyecto

El proyecto se divide en 10 fases claramente definidas. Actualmente nos encontramos con las fases 0–5 completadas y la Fase 6 en progreso:

Fase	Nombre	Estado
FASE 0	Investigación y Diseño	COMPLETADA
FASE 1	Configuración Inicial	COMPLETADA
FASE 2	Backend Base	COMPLETADA
FASE 3	Base de Datos	COMPLETADA
FASE 4	Infraestructura Docker	COMPLETADA
FASE 5	Frontend Base	COMPLETADA
FASE 6	Integración API-Frontend	EN PROGRESO
FASE 7	Módulos de Negocio	PENDIENTE
FASE 8	Testing y QA	PENDIENTE
FASE 9	Producción y Escalado	PENDIENTE

2.2 Hitos Alcanzados en Fase 4

La Fase 4 marcó el establecimiento de toda la infraestructura de contenedores. A continuación se listan los 8 hitos completados en esta fase:

#	Hito	Descripción	Estado
1	**Estructura de Carpetas**	49 carpetas creadas con arquitectura Clean/DDD bien organizada	LISTO
2	**Dockerización**	Dockerfiles optimizados para backend y frontend (multi-stage)	LISTO
3	**Orquestación**	Docker Compose con 7 servicios interconectados mediante redes	LISTO
4	**Base de Datos**	PostgreSQL 15 + Redis 7 con health checks activos	LISTO
5	**Backend**	FastAPI funcionando con rutas base y documentación Swagger	LISTO
6	**Frontend**	React + Vite configurado con hot reload para desarrollo ágil	LISTO

7	**Monitoreo**	Grafana + Prometheus instalados y datasource preconfigurado	LISTO
8	**Scripts**	Comandos de automatización sgdd.ps1 para operación rápida	LISTO

3. ARQUITECTURA DEL SISTEMA

3.1 Diagrama Conceptual de Arquitectura

La arquitectura del sistema sigue el patrón de microservicios contenerizados, con separación clara de responsabilidades por capas:

```
                    ┌──────────────────────────────────────┐
                    │          INTERNET / USUARIO          │
                    └──────────────────────────────────────┘
                                      │ HTTP :80
                                      ▼
                    ┌──────────────────────────────────────┐
                    │                NGINX                 │
                    │           Proxy Reverso (:80)        │
                    └──────────────────────────────────────┘
                                      │
                                      │
              ┌───────────────────────┼───────────────────────┐
              │ :5173                 │ :8000                 │ :3000/:9090
              ▼                       ▼                       ▼
   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
   │     FRONTEND     │   │     BACKEND      │   │     MONITOREO    │
   │  React 18 + Vite │   │   FastAPI (ASGI) │   │ Prometheus / Grafana│
   └──────────────────┘   └──────────────────┘   └──────────────────┘
                                      │
                                      │
              ┌───────────────────────┼───────────────────────┐
              │ :5432                 │ :6379                 │ Volúmenes
              ▼                       ▼                       ▼
   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
   │   POSTGRESQL 15  │   │   REDIS 7        │   │   VOLÚMENES      │
   │   Base de Datos  │   │   Caché/Sesión   │   │   Persistentes   │
   └──────────────────┘   └──────────────────┘   └──────────────────┘
```

3.2 Redes Internas de Docker

El sistema utiliza cuatro redes Docker separadas para garantizar aislamiento entre capas:

Red	Propósito	Servicios Miembros
red-frontend	Capa de presentación	frontend, nginx
red-backend	Lógica de negocio	backend, redis
red-base-datos	Persistencia	postgres, backend
red-monitoreo	Observabilidad	prometheus, grafana, backend

3.3 Volúmenes Persistentes

Los datos críticos del sistema se almacenan en volúmenes Docker persistentes que sobreviven reinicios y recreaciones de contenedores:

Nombre del Volumen	Punto de Montaje	Propósito
sgdd-postgres-data	/var/lib/postgresql/data	Datos de PostgreSQL, tablas e índices
sgdd-redis-data	/data	Snapshots y AOF de Redis
sgdd-prometheus-data	/prometheus	Series de tiempo y métricas históricas
sgdd-grafana-data	/var/lib/grafana	Dashboards guardados y configuración

4. COMPONENTES IMPLEMENTADOS

4.1 Backend – FastAPI (Python 3.12)

El backend está construido sobre FastAPI, uno de los frameworks más modernos y rápidos del ecosistema Python. Sus características principales:

- Validación automática de datos con Pydantic v2
- Documentación interactiva auto-generada en /docs (Swagger UI) y /redoc
- Soporte nativo para async/await con excelente rendimiento bajo carga
- ORM SQLAlchemy para abstracción de la base de datos relacional
- Integración con Redis para caché de respuestas y sesiones

Endpoints Funcionales Actuales:

```
GET  /              →  Mensaje de bienvenida del sistema
GET  /health        →  Health check para Docker Compose
GET  /api/v1/health →  Health check completo de la API
GET  /docs          →  Documentación Swagger (auto-generada)
GET  /redoc         →  Documentación ReDoc (auto-generada)
```

4.2 Frontend – React 18 + Vite 5.4

La interfaz de usuario está construida con el stack más moderno disponible para aplicaciones web:

- React 18.3 con soporte Concurrent Features y Suspense
- Vite 5.4 como bundler ultra-rápido (HMR en < 50ms)
- Arquitectura SPA (Single Page Application) para navegación fluida
- Nginx como servidor estático en producción (imagen multi-stage)
- Configuración separada de Dockerfile para desarrollo y producción

4.3 Base de Datos – PostgreSQL 15 + Redis 7

Parámetro	PostgreSQL 15	Redis 7
Usuario	sgdd_usuario	N/A (no auth local)
Base / DB	sgdd_bd	0 (default)
Puerto	5432	6379
Persistencia	Volumen Docker sgdd-postgres-data	appendonly=yes (AOF)

Health Check	pg_isready -U sgdd_usuario	redis-cli ping

4.4 Monitoreo – Prometheus + Grafana

La pila de monitoreo permite observabilidad completa del sistema en tiempo real:

- Prometheus recolecta métricas cada 15 segundos de todos los servicios
- Grafana visualiza las métricas con dashboards interactivos
- Datasource de Prometheus preconfigurado automáticamente en Grafana
- Credenciales de acceso a Grafana: usuario admin / contraseña admin
- Targets configurados: backend (:8000), postgres, redis

4.5 Proxy – Nginx

Nginx actúa como punto de entrada único al sistema, cumpliendo múltiples roles:

- Sirve los archivos estáticos compilados del frontend (React)
- Actúa como proxy inverso redirigiendo /api/ hacia backend:8000
- Soporte completo para SPA: redirige todas las rutas a index.html
- Configuración lista para agregar HTTPS/SSL con Let's Encrypt

```
# nginx.conf – Configuración principal
server {
    listen 80;
    root /usr/share/nginx/html;

    location / {
        try_files $uri $uri/ /index.html;  # SPA routing
    }

    location /api/ {
        proxy_pass http://backend:8000;    # Proxy al backend
    }
}
```

5. DETALLE DE LA CONSTRUCCIÓN

5.1 Estructura de Carpetas del Proyecto

El proyecto sigue una arquitectura Clean/DDD con 49 carpetas organizadas por responsabilidad:

```
SISTEMA-SAAS-DULCERIA/
├── .github/
│   └── workflows/         → Pipelines CI/CD (GitHub Actions)
├── base_datos/
│   ├── migraciones/       → Scripts Alembic de migración
│   └── semillas/          → Datos iniciales (seed data)
├── cliente/               → Frontend React
│   ├── Dockerfile
│   ├── Dockerfile.dev
│   ├── nginx.conf
│   ├── package.json
│   ├── index.html
│   └── src/
│       ├── main.jsx       → Punto de entrada React
│       ├── componentes/   → Componentes reutilizables
│       ├── paginas/       → Vistas/páginas de la SPA
│       └── servicios/     → Llamadas a la API
├── documentacion/
│   ├── diagramas/         → UML, arquitectura, flujos
│   └── srs/               → Documento de requisitos
├── infraestructura/
│   ├── docker-compose.yml → Orquestación producción
│   ├── docker-compose.dev.yml → Orquestación desarrollo
│   ├── nginx/nginx.conf   → Configuración Nginx
│   └── monitoreo/
│       ├── prometheus.yml → Config Prometheus
│       └── grafana/       → Provisioning dashboards
├── pruebas/
│   ├── unitarias/         → Tests unitarios (pytest)
│   └── integracion/       → Tests de integración
├── scripts/
│   └── sgdd.ps1           → Script principal PowerShell
└── servidor/              → Backend FastAPI
    ├── Dockerfile
    ├── requisitos.txt     → Dependencias Python
    └── app/
        ├── main.py        → Punto de entrada FastAPI
        ├── api/           → Routers y endpoints
        ├── modelos/       → Modelos SQLAlchemy
        ├── esquemas/      → Schemas Pydantic
        └── servicios/     → Lógica de negocio
```

5.2 Dockerización Detallada

Dockerfile del Backend (Python 3.12):

```
# servidor/Dockerfile
FROM python:3.12-slim

RUN apt-get update && apt-get install -y \
    build-essential libpq-dev curl && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requisitos.txt .
RUN pip install --no-cache-dir -r requisitos.txt

COPY . .

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile del Frontend (Multi-stage):

```
# cliente/Dockerfile - Multi-stage build

# Etapa 1: Compilación (Node.js)
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Etapa 2: Servidor de producción (Nginx)
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

5.3 Resolución de Problemas Documentados

Durante la construcción se encontraron y resolvieron los siguientes desafíos técnicos:

Problema Encontrado	Solución Aplicada	Lección Aprendida
pg_config: not found	Instalar libpq-dev en Dockerfile	Las dependencias del sistema operativo son críticas para módulos nativos de Python

npm: not found en prod	Usar imagen node:18-alpine solo en build stage	El multi-stage build evita conflictos de herramientas entre etapas
Python 3.14 incompatible	Bajar versión a Python 3.12-slim	Siempre usar versiones LTS estables en producción y desarrollo
Frontend unhealthy	Separar Dockerfile.dev (hot reload) del Dockerfile prod	Cada entorno (dev/prod) requiere su propia configuración Docker
Backend unhealthy	Instalar curl y agregar HEALTHCHECK con curl -f	El health check requiere que las herramientas que usa estén instaladas

6. CONFIGURACIÓN DE INFRAESTRUCTURA

6.1 Variables de Entorno (.env)

El archivo .env centraliza toda la configuración sensible del sistema. Nunca debe ser commiteado al repositorio:

```
# .env - Configuración del entorno de desarrollo
ENVIRONMENT=desarrollo
DEBUG=True

# Base de datos
POSTGRES_USER=sgdd_usuario
POSTGRES_PASSWORD=sgdd_contraseña_segura
POSTGRES_DB=sgdd_bd
DATABASE_URL=postgresql://sgdd_usuario:sgdd_contraseña@postgres:5432/sgdd_bd

# Redis
REDIS_URL=redis://redis:6379/0

# Seguridad
JWT_SECRET=clave-secreta-minimo-32-caracteres-local
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=60
```

6.2 Configuración de Prometheus

```
# infraestructura/monitoreo/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'backend'
    static_configs:
      - targets: ['backend:8000']

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres:5432']

  - job_name: 'redis'
    static_configs:
      - targets: ['redis:6379']
```

7. COMANDOS DE OPERACIÓN

7.1 Script Principal sgdd.ps1

El script principal permite controlar todo el sistema desde una sola interfaz. Todos los comandos deben ejecutarse desde la raíz del proyecto:

Comando	Acción
.\sgdd.ps1 prender	Inicia todos los servicios en orden correcto (DB → Backend → Frontend → Nginx)
.\sgdd.ps1 apagar	Detiene todos los contenedores y libera recursos del sistema
.\sgdd.ps1 status	Muestra el estado de salud de cada contenedor en tiempo real
.\sgdd.ps1 logs	Transmite logs en vivo de todos los servicios al terminal
.\sgdd.ps1 reiniciar	Apaga y vuelve a encender todos los servicios (útil tras cambios de config)

7.2 Comandos Manuales de Docker

```
# Ver todos los contenedores con su estado
docker compose -f infraestructura\docker-compose.yml ps -a

# Ver logs en tiempo real de un servicio específico
docker compose -f infraestructura\docker-compose.yml logs -f backend

# Ejecutar comando interactivo dentro del contenedor
docker compose -f infraestructura\docker-compose.yml exec backend bash

# Reconstruir imagen sin caché (tras cambios en Dockerfile)
docker compose -f infraestructura\docker-compose.yml build --no-cache backend

# Ver uso de recursos en tiempo real
docker stats
```

7.3 Accesos Rápidos del Sistema

```
# Abrir frontend en el navegador
start http://localhost:5173

# Ver documentación interactiva de la API (Swagger UI)
start http://localhost:8000/docs

# Ver documentación alternativa (ReDoc)
```

```
start http://localhost:8000/redoc

# Verificar health check del backend
curl http://localhost:8000/health

# Abrir dashboards de Grafana (admin / admin)
start http://localhost:3000

# Ver targets de Prometheus
start http://localhost:9090/targets
```

8. PRÓXIMOS PASOS Y HOJA DE RUTA

8.1 Fase 6 – Integración API-Frontend (Inmediato)

Tareas a realizar en la siguiente fase activa:

Tarea	Prioridad	Tiempo Estimado	Estado
Conectar frontend con backend (Axios)	ALTA	2 horas	PENDIENTE
Crear página de Login con validación	ALTA	3 horas	PENDIENTE
Implementar autenticación JWT completa	ALTA	4 horas	PENDIENTE
Dashboard con estadísticas del negocio	MEDIA	3 horas	PENDIENTE
CRUD completo de productos de dulcería	MEDIA	4 horas	PENDIENTE

8.2 Fase 7 – Módulos de Negocio (Corto Plazo)

- Módulo de inventario con método FIFO (primero en entrar, primero en salir)
- Punto de venta (POS) con integración de impresión de tiquetes
- Reportes con Recharts (ventas, inventario, tendencias)
- Módulo de clientes con historial de compras
- Soporte multi-tenant para múltiples tiendas bajo el mismo sistema

8.3 Fase 8 – Testing y QA (Mediano Plazo)

- Pruebas unitarias con pytest (cobertura objetivo > 70%)
- Pruebas de integración para todos los endpoints de la API
- Pruebas de carga con Locust (simular N usuarios concurrentes)
- Pruebas de regresión automatizadas en CI/CD con GitHub Actions

8.4 Fase 9 – Producción y Escalado (Largo Plazo)

- HTTPS con certificados SSL automáticos de Let's Encrypt
- Dominio personalizado y configuración DNS
- Backups automáticos de PostgreSQL a S3 o almacenamiento en la nube
- Escalado horizontal con Docker Swarm o migración a Kubernetes
- CDN para archivos estáticos y mejora de tiempos de carga globales

9. ANEXOS TÉCNICOS

Anexo A – Tabla Completa de Puertos del Sistema

Puerto	Servicio	Protocolo	Descripción de Uso
:80	Nginx	HTTP	Punto de entrada principal al sistema
:5173	Frontend Dev (Vite)	HTTP	Servidor de desarrollo React con HMR
:5432	PostgreSQL	TCP	Base de datos relacional principal
:6379	Redis	TCP	Caché en memoria y almacén de sesiones
:8000	Backend FastAPI	HTTP	API REST principal del sistema
:3000	Grafana	HTTP	Dashboards de visualización de métricas
:9090	Prometheus	HTTP	Recolector y almacén de métricas time-series

Anexo B – Comandos de Diagnóstico

```
# Filtrar solo los errores en los logs
docker compose logs backend | findstr ERROR
docker compose logs backend | findstr CRITICAL

# Estadísticas de uso de recursos en tiempo real
docker stats --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'

# Inspeccionar la red Docker del sistema
docker network inspect sgdd_red-backend

# Listar todos los volúmenes activos
docker volume ls | grep sgdd

# Conectar a PostgreSQL desde el contenedor
docker compose exec postgres psql -U sgdd_usuario -d sgdd_bd

# Conectar a Redis CLI
docker compose exec redis redis-cli
```

Anexo C – Guía de Solución de Problemas Comunes

Síntoma / Error	Solución Recomendada
port already allocated	Cambiar el puerto en docker-compose.yml o identificar y cerrar el proceso que ocupa ese puerto con netstat -ano
Cannot connect to Docker daemon	Asegurarse de que Docker Desktop esté iniciado y corriendo en la bandeja del sistema
no such file or directory	Verificar la ruta absoluta del archivo. Ejecutar desde la carpeta raíz del proyecto
permission denied (PowerShell)	Ejecutar PowerShell como Administrador. Verificar la política de ejecución con Get-ExecutionPolicy
Container status: unhealthy	Revisar logs del contenedor con docker logs <nombre>. El health check está fallando, verificar la URL configurada
ModuleNotFoundError en Python	Reconstruir la imagen con --no-cache para asegurar que todas las dependencias de requisitos.txt se instalen

CONCLUSIÓN

RESUMEN DE LOGROS ALCANZADOS

Hemos establecido una base técnica sólida y profesional para el Sistema de Gestión de Dulcerías SaaS.

La infraestructura actual es 100% funcional, probada, documentada y automatizada. Sigue buenas prácticas de Docker y Clean Architecture, es escalable y está preparada para recibir toda la lógica de negocio específica de dulcerías.

El sistema está preparado para gestionar productos como Trululu, Supercoco, Oka Loka y Barrilete, y tiene el potencial de convertirse en un producto comercializable para dulcerías colombianas.

Documento generado el 09 de Marzo de 2026 | SISTEMA-SAAS-DULCERIA

Tecnología al servicio de las dulcerías colombianas